



FACULTY OF NATURAL, MATHEMATICAL, AND
ENGINEERING SCIENCES
DEPARTMENT OF INFORMATICS

Software Engineering Group Project

Lunar

Dream Team

Aarushi Singh

Alisha Kazmi

Deeti Babbar

Extreme Limbu

Frederick George Brooke

Joanna Rachel Jayachandran

Karim He

Sakar Rai

Sadhna Arun

April 15, 2026

Contents

1	Introduction	1
2	Product	2
2.1	Objectives, context and key stakeholders	2
2.2	Requirements and specifications	2
2.2.1	Functional Requirements	3
2.2.2	Non-Functional Requirements	6
3	Design	7
3.1	System Overview	7
3.1.1	Core User	7
3.1.2	Task and Productivity	7
3.1.3	Socials and Messaging	8
3.1.4	Groups and Modules	8
3.1.5	Admin and Moderation	8
3.2	UML Diagram	9
3.2.1	Core Users	9
3.2.2	Tasks and Productivity	9
3.2.3	Social System	10
3.2.4	Group and Module	10
3.2.5	Admin and Moderation	11
3.3	Components Diagram	11
3.3.1	Groups System	11
3.3.2	Module System	11
3.3.3	Profile System	12
3.3.4	Calendar System	12
3.3.5	Map System	12
3.3.6	User Search	13
3.3.7	Tasks System	13
3.3.8	Wellbeing System	13
4	Testing	14
4.1	Overview	14
4.2	Automated Testing	14
4.2.1	Tools Used	14
4.2.2	What Was Tested	15
4.3	Manual Testing	17
4.3.1	What Manual Testing Was Used For	17

4.3.2	To What Extent Did We Rely on Manual Testing	18
4.3.3	Limits of Automated and Manual Testing	18
5	Project Management	19
5.1	Overview	19
5.2	Methodology	19
5.3	Team Organisation	19
5.4	Communication	20
5.5	What Went Well	20
5.6	What Could Have Been Improved	20
5.7	Reflections	21

Chapter 1

Introduction

Our application has been developed to address the real-world problem that users face with time management. Ineffective time management could lead to people struggling to organise tasks, missing deadlines and becoming overwhelmed from the combination of work, study and personal tasks. The aim of this application is to help users organise their time, thereby improving productivity and reducing stress. As a result, users would gain better control over their schedule, clearer prioritisation of tasks and a better work–life balance. This would support students or professionals managing multiple responsibilities.

The application was built using a modern full-stack technology stack. The frontend was developed with Next.js and TypeScript, utilising Tailwind CSS for styling and shadcn/ui as the component library to provide a consistent and accessible user interface. User authentication is handled via NextAuth.js, with Google APIs integrated to support Google account sign-in, providing a seamless and secure login experience. On the backend, Prisma serves as the ORM to interface with a MongoDB database, enabling efficient and type-safe data management. The application is deployed on Vercel, ensuring reliable cloud hosting and continuous deployment. Development dependencies include npm for package management via package-lock.json, and Jest for automated testing to maintain code quality and application reliability throughout the development process.

Chapter 2

Product

2.1 Objectives, context and key stakeholders

Our overarching objective for this project was to develop a time management system that higher education students are motivated by and not micromanaged. Several study time management applications are incredibly confusing and technical to use, whilst others add unwanted stress to students if they do not achieve their goals. We wanted to develop a time management system that was designed with students and their lives in mind. Lunar provides the ultimate goal: methodical, flexible academic planning which works around and with personal plans.

Our overarching objective can be divided into two sub-objectives:

1. Students should be able to plan and manage their academic tasks and exams.
2. Students should be able to plan and manage their personal commitments.

2.2 Requirements and specifications

The requirements for our project can be divided into functional and non-functional, and further divided by each page in our project:

Functional Requirements

- Dashboard
- Messages
- Tasks
- Calendar
- Exam Planner
- Profile
- Map
- Leaderboard

- Modules

Non-Functional Requirements

- Admin
- Notifications
- Settings

2.2.1 Functional Requirements

Dashboard

Relates to objective 1.

- The system shall display a list upcoming tasks, exams, events.
- The system shall display the leader board.
- The system shall provide a method to easily create a task from the dashboard.
- The system shall display the user's progress stats on the dashboard.
- The system shall calculate and display the user's task completion progress.
- The system shall provide the option for the user to connect to their Google Calendar from the dashboard.
- The system shall take the user to the tasks page when they click on a task on the dashboard.
- The system shall take the user to the exam's hub page when they click on an exam on the dashboard.

Tasks

Relates to objective 1.

- The system shall calculate and display the user's task completion progress.
- The system shall provide a method for the user to create, view, update and delete tasks.
- The system shall allow the user to specify details about a task, including duration, due date and priority.
- The system shall provide a search bar for the user to search through tasks.
- The system shall provide users with an organisation system to move tasks between completion status columns.
- The system shall provide a list of 'Overdue' tasks to the user.

Calendar

Relates to objective 1. and 2.

- The system shall allow users to create, schedule, view and delete events on their calendar.
- The system shall allow users to view their exams on their calendar.
- The system shall allow users to filter their events by category.
- The system shall allow users to schedule any existing tasks which are unscheduled.

Exam Planner

Relates to objective 1.

- The system shall allow users to create, update, view and delete exams.
- The system shall provide a progress bar for each exam based on task completion statistics.
- The system shall provide a method to plan exam revision by topic.
- The system shall provide a way to plan exam revision around unavailable days and time limits per day.
- The system shall create a revision plan for an exam using tasks and provide deadlines for each part of the revision.

Modules

Relates to objective 1.

- The system shall provide a module feature for a user (module lead) to create a module.
- The system shall allow users to join a module using a module pin
- The system allows module owners to create tasks and events associated to a module so that all members of the module can see these tasks and events but not edit them.

Messages

Relates to objective 2.

- The system shall provide a messaging feature to the user.
- The system shall provide a group chat feature to the user.

Leaderboard

Relates to objective 1. and 2.

- The system displays the user's and their friends' progress statistics.
- The system provides the mechanism to sort and filter the statistics on the leader board by time duration and statistic type.

Map

Relates to objective 1. and 2.

- The system shall display events and their locations on the map.
- The system shall present the user's friends' location on the map.
- The system shall show the route for the user's event and the duration of the journey.
- The system shall allow the user to add and remove saved locations.

Groups

Relates to objective 1. and 2.

- The system allows users to create, update and view groups with their friends for academic or personal use.
- The system allows a user to create tasks and events which is shared to the rest of the group members

Shop

Relates to objective 2.

- The system provides space-themed profile pictures for the user to purchase using coins they earn by completing tasks.

Mini Games

Relates to objective 2.

- The system provides a space-themed game for users to play using the coins they earn by completing tasks.

Profile

Relates to objective 1. and 2.

- The system shall calculate and display streak and task performance statistics.
- The system shall display your points and coins earned through task completion.
- The system shall provide the option to update the user's name and profile bio.
- The system shall provide a list of your friends.
- The system shall provide a list of pending friend requests for the user to accept or decline.

Preferences Quiz

Relates to objective 1. and 2.

- The system shall provide the user with a quiz where they can provide their preferences for working hours/days, reminders and prioritisation of tasks.

- The system shall use these preferences when scheduling tasks in the calendar so that the user has better control over the personal and academic time.

2.2.2 Non-Functional Requirements

Admin

Non-functional requirement.

- The system displays reports created by other users against a user.
- The system allows admins to view and carry out actions on reports raised.
- The system allows admin users to block the accounts of other users.
- The system allows only the admin user to access the admin page.

Notifications

Non-functional requirement

- The system displays notifications in a pop up box.
- The system displays notifications on the screen.
- The system provides the option to delete individual or all notifications.

Settings

Non-functional requirement

- The system allows the user to view and modify their account details.
- The system allows the user to view and modify their preferences.
- The system allows the user to update their account password.
- The system allows the user to delete their account.
- The system allows the user to connect to their Google Account.

Chapter 3

Design

3.1 System Overview

Our product is a web-based productivity app that at its crux was responsible for helping higher-education students stay in control of their busy lifestyle in an engaging format with task management, scheduling, and academic planning. It integrates advanced organisational tools with social and collaborative planning allowing users to share tasks, message and participate in group activities.

Architecturally we have chosen a full-stack approach in Next.js that uses React for the frontend UI components, server side API routes for individual application logic, and MongoDB database accessed via a PRISMA schema. For efficient testing and understanding, we split our entire system into subcomponents via decomposition with self-contained logic: Core User, Tasks and Productivity, Social, Group and Module, and Admin Moderation.

3.1.1 Core User

- This subsystem is responsible for managing user identity, authentication, personalisation, and progression within the platform.
- Where new/returning users can log back in or create an account through the login-page, furthermore for complete functionality the user can reset through an emailed link or change their password.
- Our dashboard was retrospectively designed as the main hub that a user would be in, therefore it incorporated a lot of different summary elements such as the 'to-do-list' for the day, statistics and live rankings for them to start the day with.
- We initially had more but realised that too much content on one page took too much processing and might be overstimulating for a user, therefore we refactored components out and included a 'Wellbeing' page with a reminder timer and motivational quotes to help the user.

3.1.2 Task and Productivity

- The Tasks page is responsible for displaying the number of active tasks to do in a Kanban style approach with columns to order tasks by importance and progress -

easy to visualise what the user needs to do next.

- Exam planner is a key feature that allows users to input their exam timetables with specified goal/targets that will be accounted for, and will be automatically updated to the tasks and calendar.
- The Calendar page contains a synced up list of all important dates, events and tasks, which use colour badges and filter by day/week/month to allow users to see all of their commitments in one go.
- Additionally anyone can link their google calendars or import any other calendar via the URL, for synchronisation regardless of compatibility.

3.1.3 Socials and Messaging

- Studying and revision can be isolating, so we included a friendship system to allow different users to share their own tasks, communicate and work collaboratively together for improved focus.
- Reports can then be made to handle poor behaviour as a result of adding in these social features.
- In turn notifications plays a huge part in our system as new alerts are created for each request or message that gets generated, as a single system that can be integrated into other features.

3.1.4 Groups and Modules

- The Group and Module subsystems enables collaborative planning between users through shared group spaces.
- Users can create or join groups, within which all members have access to a shared task board, a shared event calendar, and group settings.
- Groups are the central entity of this subsystem. Each group maintains a list of members, a shared task board, and a set of group events.
- When a group event is created or edited via the GroupEventModal, it is distributed to every member's personal calendar automatically through a server-side cascade, ensuring all members remain synchronised without manual intervention.

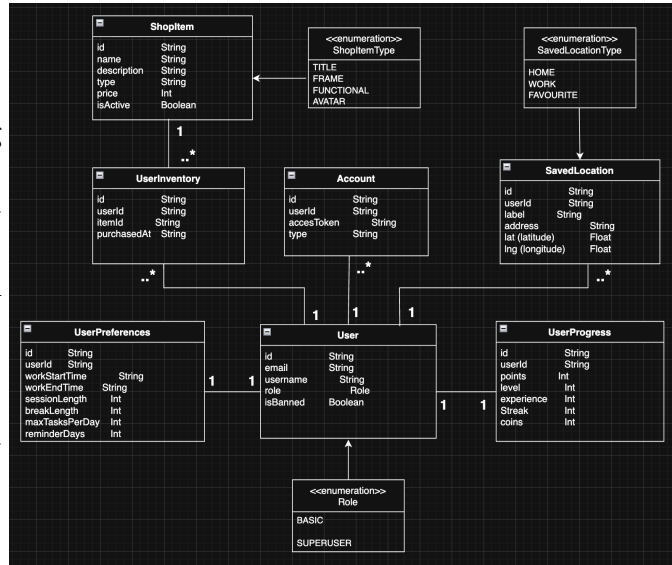
3.1.5 Admin and Moderation

- Adding in social elements with multi-user interaction will inevitably bring poor behaviour, which is why we have added an 'Admin' superuser feature that grants moderators the ability to view a summary of all the information and handle reports/appeals.
- The page was constructed using tabs to store and access the list of appeals, users and reports all of which reuse the same filter and view panels.
- Reports that are made on messages or users will display on the list of reports and are marked as 'PENDING', which an admin can view and either temporary ban as a warning, ban or reject the report.

3.2 UML Diagram

3.2.1 Core Users

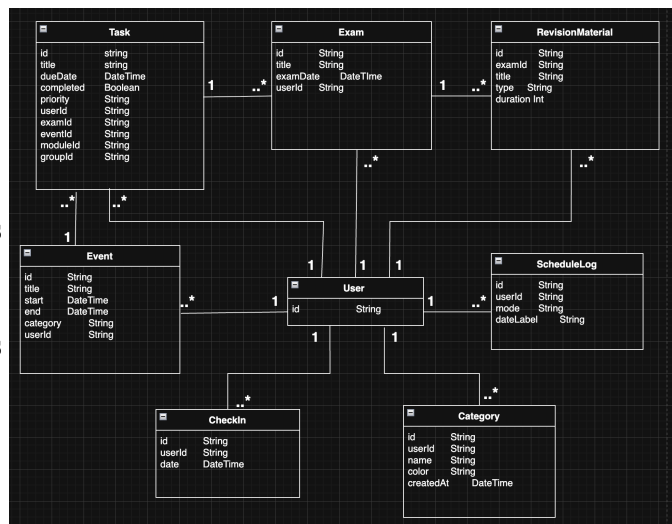
- User - central entity
- Account - authentication using OAuth
- UserPreferences - scheduling behaviour
- UserProgress - gamification (points, level, streaks)
- UserInventory - purchased items
- ShopItem - available items
- SavedLocation - map-based features



This subsystem is designed with the User as the central entity, with all supporting entities extending functionality directly from it. A one-to-one relationship is used for UserPreferences and UserProgress to ensure each user has a single configuration and progression state, whilst one-to-many relationships are used for Accounts and SavedLocations to support multiple accounts under the same user. The separation of UserInventory and ShopItem enables a scalable shop system, where items are defined independently of ownership and a complete inventory of such is kept.

3.2.2 Tasks and Productivity

- Task - central unit of work
- Exam - academic deadlines
- RevisionMaterial - study resources
- Event - scheduled activities
- Category - task organisation
- ScheduleLog - generated schedules
- CheckIn - daily tracking

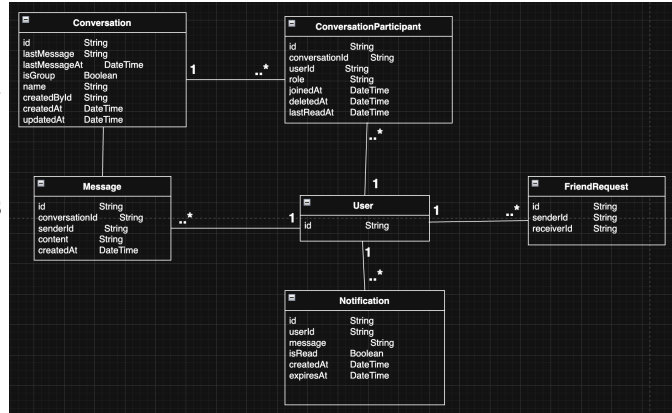


The Task entity is the core component here by acting as the primary unit of work. Tasks are designed to optionally associate with Exams and Events allowing for flexibility between structured academic planning and general scheduling. Supporting entities such as RevisionMaterial enhance the academic functionality, while ScheduleLog and CheckIn provide mechanisms for tracking and analysing user productivity. Optional relationships are used to avoid over-constraining tasks, allowing users to create both standalone and

structured tasks.

3.2.3 Social System

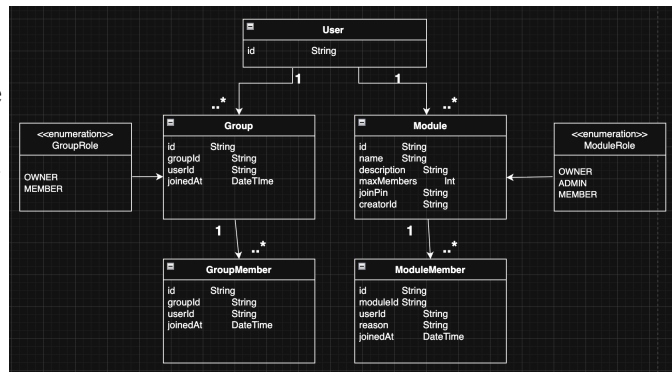
- FriendRequest - manages friend-ships
- Conversation - chat container
- ConversationParticipant - links users to conversations
- Message - communication
- Notification - system alerts



The system uses an intermediary entity(ConversationParticipant) to model the many-to-many relationship between users and conversations, allowing support for both private and group messaging. FriendRequest models the lifecycle of friendships, including pending, accepted, and rejected states. Notifications are decoupled from other entities, which allows the system to generate alerts from multiple sources without tight coupling.

3.2.4 Group and Module

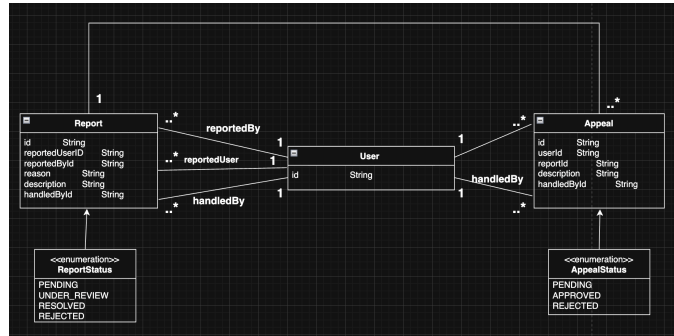
- Module - structured collaborative unit (e.g. course)
- ModuleMember - membership + roles
- Group - informal collaboration
- GroupMember - membership



Many-to-many relationships between users and collaborative entities are resolved using intermediary models (ModuleMember and GroupMember), which also store role information. This design supports role-based permissions (owner, admin, member) and ensures scalability for group-based features. The separation between Module and Group allows differentiation between structured (academic) and informal collaboration.

3.2.5 Admin and Moderation

- Report - user-submitted issue
- Appeal - dispute of a report
- User - involved in multiple roles



The Report entity maintains multiple relationships with the User entity to represent different roles in the moderation process. These relationships are explicitly labelled to avoid ambiguity. Appeals are linked to Reports to allow users to challenge moderation decisions, supporting fairness and accountability.

3.3 Components Diagram

3.3.1 Groups System

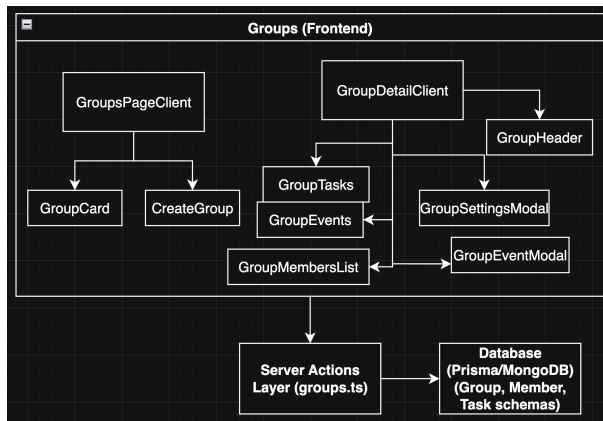


Figure 3.1: Groups component diagram

The diagram shows the frontend structure for the Groups feature, with main containers (GroupsPageClient, GroupDetailClient) coordinating sub-components for tasks, events, members, and settings

3.3.2 Module System

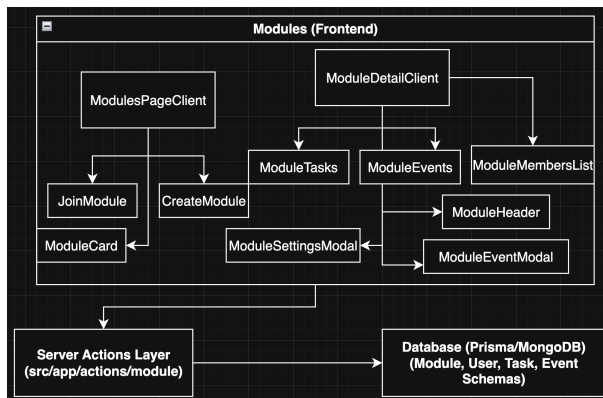
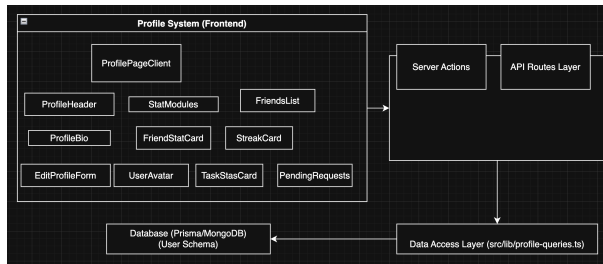


Figure 3.2: Module component diagram

Users interact and can join shared modules with other users using these modularised components.

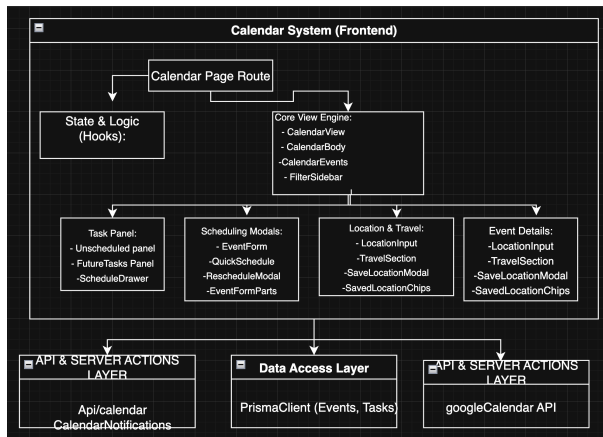
3.3.3 Profile System



Displaying the profiles for users along with their data (friends, scores, and points), as well as offering editing, sending friend requests, or reporting another user.

Figure 3.3: Profile component diagram

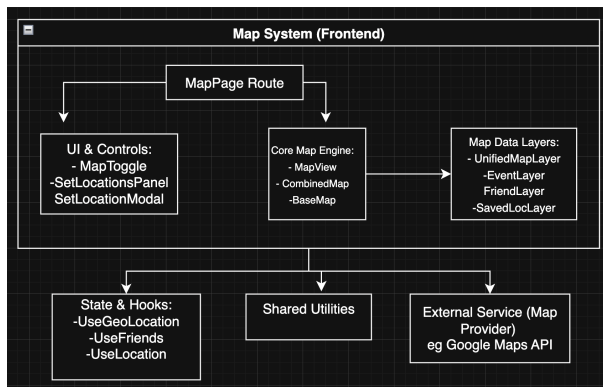
3.3.4 Calendar System



Personalised events, exams, and tasks can be added to a user's timetable to support studying and set deadlines or goals.

Figure 3.4: Calendar component diagram

3.3.5 Map System



View locations of tasks and events, calculate estimated travel time, and display friend locations as map pins.

Figure 3.5: Map component diagram

3.3.6 User Search

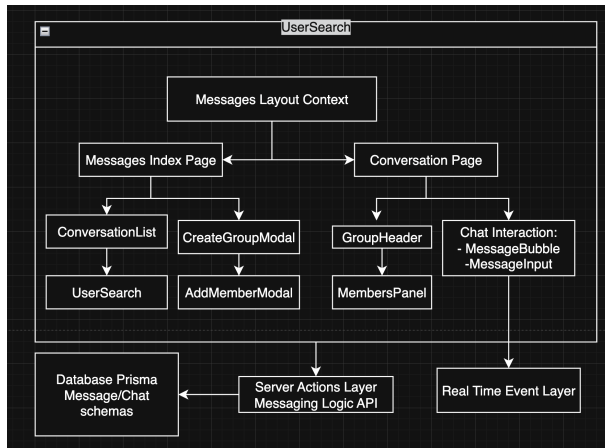


Figure 3.6: User search component diagram

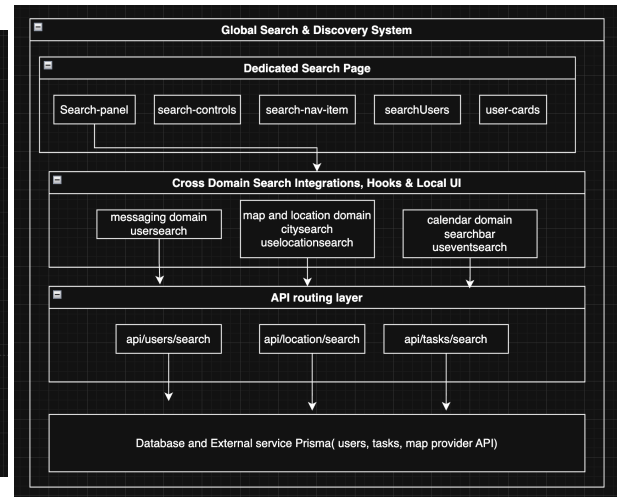


Figure 3.7: User search component diagram

3.3.7 Tasks System

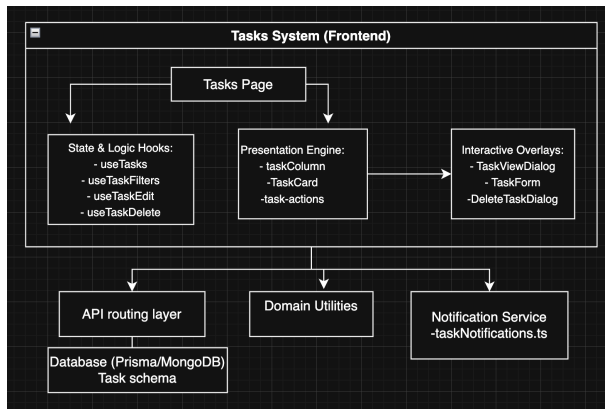


Figure 3.8: Tasks component diagram

Modularised layout of the To-do-list and allocation of tasks within the calendar and dashboard feature

3.3.8 Wellbeing System

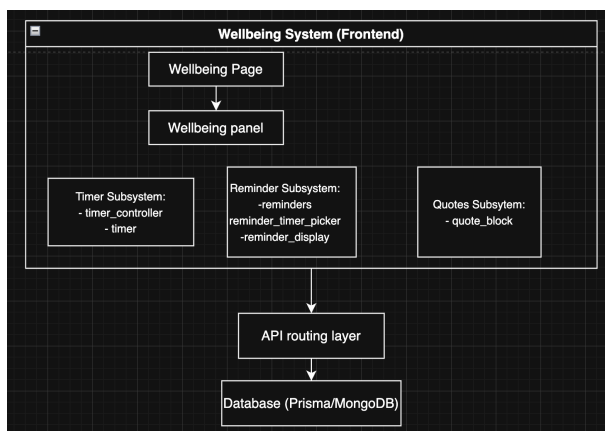


Figure 3.9: Wellbeing component diagram

Provides focus timers and reminder features to support user wellbeing and productivity.

Chapter 4

Testing

4.1 Overview

The team used automated testing as the main way of checking the application worked correctly. Tests were written by all team members using Jest, with React Testing Library used where tests needed to check how React components and hooks behave. Tests are stored alongside the code they test and were run locally during development.

The test suite covers a wide range of the application, including page components, API endpoints, custom hooks, and business logic. Manual testing was also used alongside automation, mainly for features that are visual or interactive and harder to check with code alone.

4.2 Automated Testing

4.2.1 Tools Used

All automated tests use Jest as the test runner. React Testing Library is used on top of this to render components and hooks in a test environment. Where tests need to check Next.js-specific behaviour — such as authentication, redirects, or server responses — those parts are replaced with mocks so the test stays focused on the logic being checked.

Coverage was measured using Jest’s built-in coverage tool. The results across the full codebase are shown in Table 4.1.

Metric	Covered / Total	Coverage
Statements	10,031 / 10,239	97.96%
Branches	4,411 / 4,765	92.57%
Functions	2,593 / 2,665	97.29%
Lines	9,109 / 9,236	98.62%

Table 4.1: Jest coverage results across the full codebase.

Statement, function, and line coverage are all above 97%, meaning the vast majority of the codebase is exercised by the automated tests. Branch coverage is slightly lower at 92.57%,

though this still represents strong coverage across the application. The remaining untested branches are mostly found in interactive UI components where behaviour depends on user input or visual state, and in edge cases within complex multi-step flows that are better suited to manual testing.

4.2.2 What Was Tested

Tests were written across every major area of the application.

Authentication and Login

The login, registration, forgot password, and reset password pages are all tested. Tests confirm that users are redirected when not signed in, that the login flow behaves correctly for both valid and invalid credentials, and that registration and password reset flows handle both success and error cases.

Dashboard

The dashboard page tests check that the correct content is shown to a logged-in user, including upcoming tasks and progress statistics. The coming up soon component is tested to confirm it filters and displays tasks due within the next seven days correctly.

Tasks

Several test files cover the Tasks feature. These check that task cards display the correct information, that the to-do list renders and updates correctly, and that the tasks page behaves as expected. A custom hook that manages task state is also tested separately.

Calendar

The calendar tests cover the event form and its individual sections, including the travel section. Tests check that form fields work correctly, that events are created and updated as expected, and that the component renders correctly across its different states. The calendar import flow and travel preview API endpoint are also tested. Due to Google OAuth testing mode, refresh tokens expire after 7 days. A reconnect button is provided to re-authenticate and restore functionality.

Exam Planner

The scheduling logic behind the Exam Planner is tested directly. This includes checking that unavailable dates are excluded, that topics are spread across days correctly when a daily revision limit is set, and that the full revision plan is generated with tasks assigned to the right dates. The exam planner page and individual exam detail view are also tested.

Messaging

Tests cover the conversation list and individual message bubbles, checking that messages are displayed correctly and that the conversation page renders as expected. Tests also cover the conversation detail, members, and typing indicator API endpoints.

Modules

The modules area has tests for the modules list page, the module detail view, the create module form, and the module event modal. These check that the pages render the right content and that user interactions work correctly.

Groups

Tests cover the create group form, the group event modal, and the group detail page. These check that these components render and behave as expected, including handling of group-specific actions.

Leaderboard

The leaderboard page is tested to confirm that statistics are displayed correctly and that filtering works as expected.

Map

The map feature is tested extensively. The geolocation hook is tested in isolation, covering the initial loading state, successful location retrieval, browser errors, unsupported browsers, and the guarantee that the browser is only asked for the location once. The `SavedLocationsPanel` component is tested across all of its states, including the header, loading state, empty state, location cards, delete, rename, and the add location form including address search, suggestion picking, type selection, and the full save lifecycle. The `SetLocationModal` component is also tested, covering rendering, geolocation initialisation, search, suggestion selection, map interaction, save success and failure, and the hidden location toggle. The location saved and location search API endpoints are tested separately.

Profile

The profile page and public profile view are tested to confirm that user information is displayed correctly. Profile actions are also tested.

Settings

The settings page tests check that user preferences can be viewed and updated correctly. The `PreferencesForm` component is tested to confirm it loads existing preferences, handles user interactions across all form fields, saves successfully, and handles save failures gracefully. The preferences API endpoints are also tested.

Admin

The admin API endpoints for reports, appeals, users, stats, and bans are all tested. These tests check that only admin users can access the endpoints, that filtering and sorting query parameters work correctly, and that pagination behaves as expected.

Shop

The shop page component is tested to confirm it renders correctly and handles its expected states.

Quiz

The quiz page is tested to confirm that questions are displayed correctly and that user interactions are handled as expected.

Games

The games page is tested to confirm that it renders correctly and that the game interactions behave as expected.

Wellbeing

The wellbeing page and its associated API endpoints are tested, including the wellbeing centre and timer endpoints.

UI Components

Core shared UI components are tested in isolation. This includes the chart components (`ChartContainer`, `ChartStyle`, `ChartTooltipContent`, and `ChartLegendContent`), dialog components, and toggle group components. These tests confirm that the components render correctly, apply the correct styles, and respond to user interaction as expected.

Hooks and Library Code

Custom hooks are tested separately from the components that use them. This includes hooks for tasks, calendar interactions, events, and geolocation. The scheduling library, calendar utility functions, and map utilities are also tested directly to confirm that business logic is correct independently of the UI.

4.3 Manual Testing

4.3.1 What Manual Testing Was Used For

Manual testing was carried out throughout development, mainly to check features that are difficult to test with code. This included checking that pages looked correct, that interactive features like third-party integrations such as Google Calendar connected properly, and the external API's that were used

The purpose of manual testing was not to replace automated tests, but to cover the gaps they leave. Anything with a clear, predictable output — such as API endpoints, scheduling logic, or authentication checks — was covered by automated tests. Manual testing was used where the feature involves real-world interactions, a browser environment, or visual judgement.

4.3.2 To What Extent Did We Rely on Manual Testing

The team relied on manual testing only for areas where automation was not practical. The automated suite covers the vast majority of the application logic, as shown by the coverage figures above. Manual testing was reserved for the following areas that could not be reasonably automated:

- **Real-time messaging** — checking that messages arrive instantly requires a live server and connected users.
- **Map interactions** — the map, route display, and friend locations depend on the browser and a live tile provider.
- **Google Calendar connection** — the sign-in flow requires a real browser and Google account.
- **Drag-and-drop on the task board** — simulating pointer drag events reliably in a test environment is not straightforward.
- **Leaderboard statistics** — fully testing the data aggregation requires a database populated with realistic activity.

4.3.3 Limits of Automated and Manual Testing

Automated testing is reliable and repeatable, but it cannot easily cover visual appearance, real-time behaviour, or interactions that depend on external services. It also requires the test to be written correctly in the first place — a poorly written test can pass even when something is broken.

Manual testing can catch issues that automation misses, particularly around look and feel, but it is less reliable overall. It depends on the tester remembering to check everything and noticing when something goes wrong. It also cannot catch problems introduced by later code changes unless the test is repeated. For these reasons, manual testing was only used where automation was not practical.

Chapter 5

Project Management

5.1 Overview

This chapter reflects on how the team organised and managed the development of Lunar across the project lifecycle. It covers the methodology we adopted, how we coordinated work, what went well, and what we would approach differently given the chance.

5.2 Methodology

We followed an Agile development methodology, working in iterative sprints to allow us to build, review, and refine features incrementally. This suited the nature of the project well — requirements evolved as we developed and tested the application, and an iterative approach gave us the flexibility to respond to those changes without significant disruption.

Work was planned and tracked using a shared project board (Trello), where tasks were created, assigned, and moved through stages from backlog to completion. Sprint planning meetings were held at the start of each cycle to agree on priorities, and review sessions at the end allowed the team to reflect on progress before moving on.

5.3 Team Organisation

Responsibilities were divided across the team based on individual strengths and areas of interest, though in practice all members contributed across the full stack. Ownership of specific subsystems was agreed early on to avoid duplication and ensure accountability. The main areas of ownership were:

- **Core User and Authentication** — account management, login flows, and user preferences.
- **Tasks and Exam Planner** — the Kanban board, scheduling logic, and exam revision planner.
- **Calendar and Map** — event scheduling, Google Calendar integration, and location features.

- **Social and Messaging** — friend requests, messaging, notifications, friend map, leaderboard.
- **Groups and Modules** — collaborative spaces, shared tasks, and module membership.
- **Admin and Moderation** — reporting, appeals, and the admin dashboard.

Despite this division, integration between subsystems required regular communication and cross-team coordination, particularly around shared data models and API contracts.

5.4 Communication

The team communicated primarily through WhatsApp for day to day updates and quick decisions. More substantial discussions, such as sprint planning, design decisions, and integration work, were handled in scheduled meetings held on a bi-weekly basis, throughout the project.

Version control was managed using Git with a shared repository. We used a branching strategy where each feature was developed on its own branch and merged into the main branch following review. This helped prevent conflicts and kept the main codebase stable throughout development.

5.5 What Went Well

The Agile approach worked effectively for this project. Breaking development into sprints meant that working features were available early, and feedback could be incorporated before too much work had been built on top of an incorrect assumption. The high automated test coverage — above 90% across all metrics — reflects a consistent effort to test code as it was written rather than treating testing as an afterthought.

Subsystem ownership also worked well in practice. Having clear responsibility for different parts of the application meant that progress was rarely blocked, and team members could work in parallel without stepping on each other's work.

5.6 What Could Have Been Improved

Integration between subsystems occasionally caused friction. While individual components were developed and tested in isolation effectively, bringing them together sometimes revealed mismatches in assumptions — for example, around the shape of shared data or the behaviour of shared components. Earlier and more frequent integration testing would have helped catch these issues sooner.

In some cases, requirements for a feature were not fully agreed upon before development began, leading to rework when the expected behaviour turned out to differ between team members. More detailed specification of edge cases and acceptance criteria at the planning stage would have reduced this.

5.7 Reflections

Overall, the team worked well together and delivered a product that fully meets the objectives set out at the start of the project. The decision to use Next.js as a unified full-stack framework reduced complexity at the integration boundary between frontend and backend, and the use of Prisma gave us a consistent and type-safe way to interact with the database throughout.

If we were to approach a similar project again, we would invest more time upfront in agreeing shared data contracts and API interfaces before parallel development began. We would also introduce integration testing earlier in the cycle rather than relying on it primarily at the end. These changes would reduce the coordination overhead that emerged mid-project and allow individual streams of work to come together more smoothly.